

Enhancing Visibility on a Science DMZ with P4-perfSONAR

Ali Mazloun^a, Elie Kfoury^a, Ali AlSabeh^b, Jose Gomez^c, Jorge Crichigno^a

^a*Molinaroli College of Engineering and Computing, University of South Carolina, Columbia, SC, U.S.A*

^b*College of Sciences and Engineering, University of South Carolina Aiken, Aiken, SC, U.S.A*

^c*Katz School of Business, Fort Lewis College, Durango, CO, U.S.A*

Abstract

The Science Demilitarized Zone (Science DMZ) is a specialized network designed to facilitate the transfer of large-scale scientific data. One of the key elements of the Science DMZ is perfSONAR, an active performance measurement device that monitors end-to-end paths over multiple domains. Although versatile, perfSONAR faces limitations such as restricted visibility of events and course-grained measurements. This paper proposes a scheme that integrates P4 programmable data plane (PDP) switches with perfSONAR. P4 PDP switches are passively installed and operate on real-time traffic copies, providing flexibility to collect fine-grained custom measurements and report events in the data plane. This integration enables perfSONAR to collect per-flow granular statistics of actual traffic, identify a broader range of networking issues, and enhance visibility while reducing the overhead of active tests. Additionally, the scheme uses an adaptive linear prediction (LP) model that dynamically adjusts the rate of reports sent from the P4 PDP switch to perfSONAR, minimizing the storage and processing needed for the latter. Experimental results show that the system reduces the number of reports by a factor of five while maintaining a small and configurable relative mean error (RME).

Keywords:

perfSONAR, Science DMZ, P4, Data Transfer Node (DTN), Bandwidth-delay Product (BDP), Router's Buffer Size, Microburst, Millimeter Wave (mmWave).

1. Introduction

In today's world of extensive scientific and engineering pursuits that generate vast amounts of data, establishing reliable network infrastructure has become paramount. From the immense data produced by cutting-edge facilities like the Large Hadron Collider [1] to the intricate genetic insights derived from portable DNA sequencers [2], the need for high-performing data transfer across networks is more pronounced than ever. While traditional enterprise networks can handle day-to-day operational data well (e.g., e-mails and Web content), they face significant challenges managing the flood of terabyte and petabyte-scale data generated by these scientific instruments.

In response to these challenges, the Science Science Demilitarized Zone (DMZ) has emerged [3] as a dedicated network segment explicitly designed for high-speed transmission of scientific data. The Science DMZ architecture relies on four essential elements: (1) specialized Data Transfer Nodes (DTNs) for swift data movement, (2) high-throughput paths ensuring seamless data flow, (3) tools for real-time network performance measurement, and (4) security protocols tailored for the unique demands of high-performance scientific environments.

A critical aspect of the Science DMZ's effectiveness lies in its advanced monitoring and analysis capabilities, which are essential for detecting and resolving performance issues. At the heart of the Science DMZ, perfSONAR [4] serves as a key tool for effectively monitoring network performance and troubleshooting. Developed through collaboration among research and education institutions, perfSONAR offers a comprehensive framework for measuring, analyzing, and diagnosing network issues. It empowers network administrators, researchers, and operators to gain valuable insights into network performance, ensuring the efficiency and reliability of their infrastructure. Despite its critical role in diagnosing network issues, perfSONAR has limitations related to the added overhead, packet-level analysis, custom network measurements, and detection of problems originating from the DTNs.

This paper extends our previous work [5] to enhance the monitoring and troubleshooting capabilities of perfSONAR by utilizing P4 programmable data planes (PDPs). P4 is a programming language for customizing the packet processing behavior of network devices, allowing network operators to define how packets are parsed, processed, and forwarded. Thus, providing high flexibility and programmability in network operations [6]. In the proposed system, a P4 PDP switch is deployed passively and receives a copy of the real traffic from optical Test Access Point (TAP). The switch provides per-flow network measurements for the number of bytes, round trip time (RTT), packet loss, and packets' interarrival time (IAT). The switch also monitors the queue occupancy on a per-packet basis and

Email addresses: amazloun@email.sc.edu (Ali Mazloun), ekfoury@email.sc.edu (Elie Kfoury), ali.alsabeh@usca.edu (Ali AlSabeh), jagomez@fortlewis.edu (Jose Gomez), jcrichigno@cec.sc.edu (Jorge Crichigno)

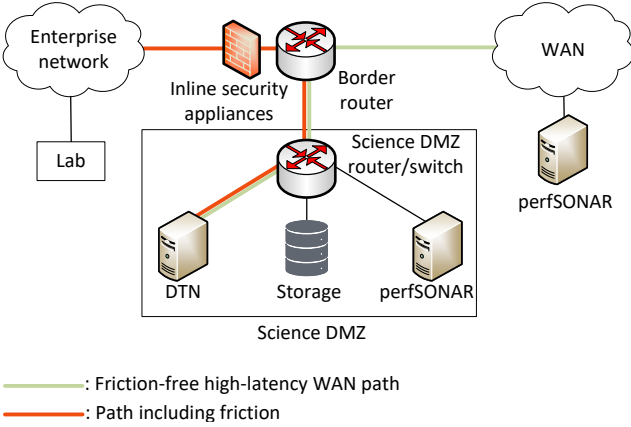


Figure 1: A Science DMZ co-located with an enterprise network.

detects and reports microbursts. The per-flow measurements are collected by the data plane and then extracted and processed by the control plane of the switch before being reported to the perfSONAR archiver.

Furthermore, the proposed system deploys an adaptive model based on the Linear Prediction (LP) technique to reduce the processing and storage overhead at the archiver. The model dynamically adjusts the reporting rate depending on the variation of the metrics. It minimizes the reporting rate under stable network conditions (i.e., stable data transfers) and increases the rate otherwise. The model runs on the control plane of the switch and selectively chooses which data plane extracted reports should be sent to the perfSONAR archiver.

1.1. Contributions

The main contributions of this paper are summarized as follows:

- Providing per-flow detailed statistics in real-time. The P4 PDP switch supports custom data processing and measurement collection at line rate.
- Achieving seamless integration between the P4 PDP switch and perfSONAR. New Applications Programming Interfaces (APIs) are defined to allow the perfSONAR configuration layer to manage the P4 PDP switch at run time, and the perfSONAR archiver is configured to collect the network measurements reported by the P4 PDP switch.
- Minimizing the processing and storage overhead on the archiver. The system deploys an adaptive model that dynamically adjusts the reporting rate from the control plane to the archiver depending on the variation in the traffic.
- Publishing the source code of the system. It can be accessed via [7].
- The system has been deployed on the production science DMZ of the University of South Carolina and is providing granular live measurements [8].

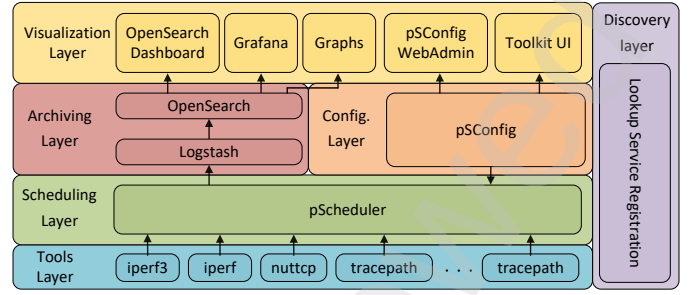


Figure 2: perfSONAR architecture [4].

2. Background

2.1. Science DMZ

General purpose networks, also referred to as enterprise networks, may require days or even weeks to transfer science data, which could be terabyte or petabyte-scale [9]. To address this challenge, the Science DMZ is proposed. The Science DMZ is a networking design dedicated to scientific data transfers. It has four key pillars: (1) optimized DTNs that support high data transfers, (2) high-throughput frictionless paths, (3) performance monitoring tools, and (4) specialized security protocols.

Figure 1 depicts the architecture of a simple Science DMZ. The path connecting the Science DMZ to the Wire Area Network (WAN) contains no CPU-intensive appliances, allowing frictionless data transfers. This is essential to minimize packet losses, especially since this path is characterized by its high latency, where losses can significantly degrade data transfer performance [3]. The path connecting the Science DMZ to the enterprise network deploys inline security appliances. Because the path has low latency, packet losses caused by the inline appliances have minimal effect on the performance of the data transfers. Thus, the users from the enterprise network experience reasonable performance when accessing the data stored in the Science DMZ.

The best practice is to directly connect the Science DMZ router/switch to the border router. If that is not possible, the number of network devices between the Science DMZ and the border router should be minimized to avoid packet losses caused by these intermediate devices. Another deployment characteristic that improves the overall performance of Science DMZ is the utilization of dual-homed DTNs. DTNs may have a secondary interface connected to the storage device, allowing them to simultaneously receive and transfer the data.

2.2. perfSONAR

perfSONAR is a key element of the Science DMZ architecture [9]. It provides the necessary visibility to locate and troubleshoot networking problems. To measure the performance of a path connecting two distinct domains, two or more perfSONAR nodes coordinate and perform active measurement tests. The main characteristic of perfSONAR is that a network engineer can schedule a test with a remote perfSONAR node without having control over the domain where it resides.

The architecture of the latest version of perfSONAR (version 5) is depicted in Figure 2. perfSONAR consists of 6 layers, which are the tools layer, the scheduling layer, the archiving layer, the configuration layer, the visualization layer, and the discovery layer. The tools layer is a collection of open-source tools. The user does not invoke these utilities directly. Instead, the pscheduler command from the scheduling layer is used. The scheduling layer consists of a single tool named pScheduler which is responsible for choosing the timing of the tests to avoid any time conflict, running the tests and collecting the results, and directing the results to the archiving layer (if an archiver is configured).

The archiving layer is responsible for storing time-series data about the tests that were performed. This layer adopts the plugin architecture, where the user can choose which software to use in this layer. The default archiver used by perfSONAR is the combination of Logstash and OpenSearch [10]. Logstash is a data processing pipeline that accepts data from multiple sources, provides flexible data processing, and forwards the processed data to the final destination [11]. OpenSearch is a search and analytical suite that supports data storage, real-time application monitoring, and log analytics [12]. The type of tests to be performed and the archiver to be used are set by the configuration layer. The primary component of this layer is called pSConfig. pSConfig automates the process of scheduling tasks between a cluster of perfSONAR nodes. As the name of the fifth layer indicates, it is used to visualize the measurements collected by the archiver. The last layer registers the existence of a perfSONAR node in a public and/or private simple lookup service. A simple lookup service is a database that stores data in a key-value pair format. A perfSONAR node also uses this layer to find hosts with which to test.

2.3. P4 Programmable Data Planes

Network devices are composed of a data plane and a control plane. The data plane, also known as the forwarding plane, is responsible for processing and forwarding the packets. The control plane populates the data plane with the forwarding rules. In traditional networking devices, the data and control planes were constrained by proprietary hardware and software architectures, limiting their flexibility and the range of operations they could perform [6, 13, 14, 15]. PDPs emerged in response to these limitations. A PDP switch or router provides the flexibility to define custom packet processing on the data plane. P4 is a widely adopted programming language for PDPs. It specifies how packet headers are parsed, what operations are to be performed on the parsed headers and their metadata, and which headers are to be serialized before forwarding the packets [16].

2.4. Adaptive Sampling

Adaptive sampling is the process of dynamically adjusting the sampling rate based on the observed measurements. Adaptive systems can be roughly classified as fuzzy logic or LP [17]. The fuzzy logic technique imitates human logic to adjust the sampling rate. It maps the current network behavior to similar

past behavior and extrapolates the future behavior accordingly. After that, the sampling rate is modified depending on the extrapolated behavior [18]. The LP technique builds a model from the observed measurements. The model is then used to predict future measurements. The sampling rate is reduced if the prediction is close enough to the observation as the model captured the traffic pattern. Otherwise, the sampling rate increases to detect the new pattern [19].

3. Motivation

perfSONAR mainly uses active tests to measure performance and locate networking issues. Using active tests has a few drawbacks. First, the induced traffic adds an overhead to the network. Performing active tests might overload the network, causing packet losses and performance degradation. Second, the measurements collected from the active tests might not represent the real traffic. One reason is that the active test might be routed through a different path than the real traffic. Furthermore, the visibility provided by perfSONAR is limited by the duration of the active tests. Thus, the current deployment of perfSONAR suffers from a trade-off between the induced network overhead and the provided visibility. Higher visibility means more active tests and, consequently, higher overhead. Using active tests also limits the visibility of perfSONAR to the segment connecting its nodes. Any device outside this segment is not monitored by perfSONAR. Thus, perfSONAR does not detect problems caused by the DTNs. Misconfiguring the DTNs (e.g., small TCP buffers) leads to significant performance degradation. The limited visibility supported by perfSONAR prevents it from locating problems caused by the DTNs.

In addition to the drawbacks related to active tests, perfSONAR is not capable of providing fine-grained measurements. perfSONAR is a software framework that utilizes software tools to collect measurements. The granularity achieved by perfSONAR is constrained by the protocols and tools it utilizes. For example, perfSONAR uses the Simple Network Management Protocol (SNMP) to collect statistics from the networking devices. However, SNMP provides coarse-grained measurements [20], preventing perfSONAR from detecting queue-related problems like microbursts. A microburst occurs when a large number of packets arrive in a short time interval. If a networking device is configured with a small buffer size, the device might fail to absorb the microburst, leading to packet losses.

One approach to address these limitations is to operate on the real traffic and on a per-packet basis [21]. If perfSONAR collects measurements from real traffic, it produces more representative statistics, providing higher visibility and reducing the overhead of active tests. More representative and accurate statistics are collected as perfSONAR operates on the real traffic rather than the traffic injected by its active tests. Higher visibility is achieved because perfSONAR monitors the performance of the network for the duration of the data transfers rather than the duration of the active tests. This eliminates the need for periodic active tests, reducing the associated overhead. However, as a software tool, perfSONAR cannot scale to the data rates of Science DMZ, preventing it from operating at line rate. This

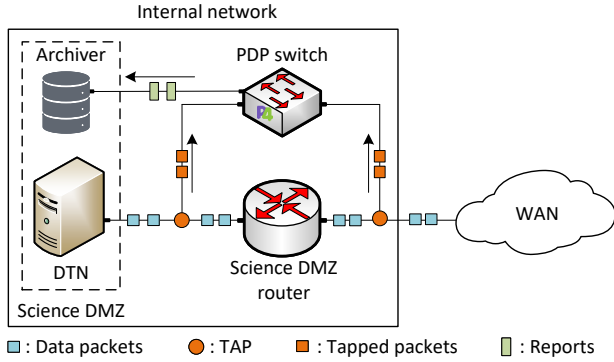


Figure 3: Proposed system overview. Fine-grained measurements are collected from the real traffic through the passive deployment of a PDP switch.

approach requires a system with hardware-based performance to perform data collection in real time and software-based flexibility to support custom measurement collection. Because PDP switches support all these features, they have the potential to complement the functionalities of perfSONAR. This paper proposes introducing PDP switches to the Science DMZ architecture to complement the functionalities of perfSONAR.

4. Proposed System

4.1. Overview

The proposed system utilizes a P4 PDP switch to operate on a copy of the real traffic and collect fine-grained measurements. As depicted in Fig. 3, two optical TAPs duplicate the traffic arriving to and departing from the Science DMZ router. The duplicated packets are forwarded to the PDP switch. The P4 PDP switch is programmed to provide per-flow statistics of the real traffic. The control plane of the switch extracts the network measurements from the data plane and selectively chooses the measurements to be reported to the archiver. The measurements are then forwarded to the perfSONAR archiver.

4.2. Measurement Collection

The data plane of the PDP switch is responsible for measurement collection. As depicted in Fig. 4, it collects per-flow number of bytes, RTT, packet losses, and packet IAT. It also measures the queuing delay of the Science DMZ router on a per-packet basis. The measurements are stored in stateful registers (stateful memory elements in the data plane) and are indexed by the identifier (ID) of the flows. A flow ID is calculated from the hash of the packets' 5-tuple (i.e., source and destination IP addresses, source and destination ports, and protocol).

The switch maintains the IDs of the active flows. When a packet arrives, its 5-tuple is hashed, and the hash value is compared to the maintained flow IDs. If the packet belongs to an active flow, the statistics of the corresponding flow are updated to reflect the new packet. The throughput of the flow is calculated from its number of bytes. For each received packet, the number of bytes is incremented by the packet length (a field in the IPv4 header).

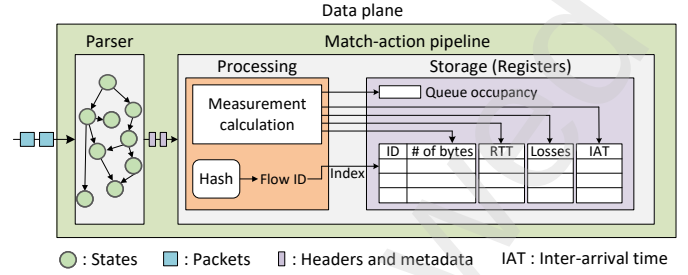


Figure 4: Measurement calculation and storage in the data plane. The data plane computes the per-flow number of bytes, RTT, packet losses, and IAT and stores them in stateful registers indexed by the flow ID. The queue occupancy is computed per Science DMZ router.

The RTT and the packet losses calculation approach is adopted from [22]. To measure the RTT of the flows, the data plane calculates the time elapsed between a data packet and its corresponding acknowledgment (ACK). Data packets are those with non-zero payloads. ACKs are control messages sent back by the receiver to acknowledge the successful receipt of data packets. The switch calculates the expected ACK number of a data packet and uses the number to index a register that stores the arrival time of the data packet. When an ACK arrives at the switch, its ACK number is compared to the register holding the expected ACK numbers. If there is a match, the timestamp stored at the cell indexed by the ACK number is retrieved and used to calculate the RTT.

The data plane maintains the per-flow packet losses by monitoring the retransmissions. Retransmissions occur when TCP segments are not acknowledged within a specific timeframe or when errors in transmission are detected. Upon detecting missing or corrupted segments, TCP retransmits those segments, ensuring data integrity and reliability. Each segment has a sequence number. The sequence number increases proportional to the number of bytes transmitted. It always increases during the lifetime of the connections. The only exceptions are caused by overflow or packet retransmission. An overflow occurs because the TCP sequence size field is only 32 bits long and can hold a finite value. When the sequence number exceeds the maximum value that can be represented in 32 bits, an overflow occurs, and the sequence number wrap-around to zero. On the other hand, when a retransmission occurs, the sequence number of the retransmitted packet will be less than the last observed sequence number. In this case, a packet loss is considered.

Queuing delay is calculated as the time packets spend inside the monitored Science DMZ router. The PDP switch receives two copies for each packet. The first copy is for the packets entering the Science DMZ router. The second copy is for the packets exiting the router. The queuing delay is then calculated as the difference in the arrival time of the two copies of the same packet.

4.3. Measurement Extraction

The control plane is responsible for extracting the measurements collected by the data plane. The control plane accesses the registers at runtime through the APIs provided by the P4

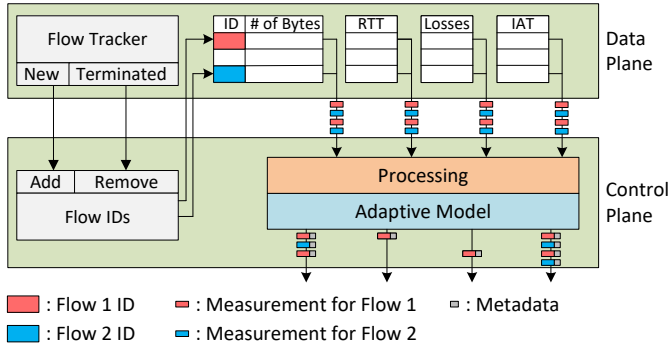


Figure 5: Measurement extraction by the control plane. The control plane maintains the list of active flows and continuously extracts and processes their measurements from the data plane.

PDP switch vendor. The first step in the extraction process is to obtain the IDs of the flows. The data plane notifies the control plane when a new flow is detected and when an existing flow terminates. The notification sent by the data plane when a new flow is detected contains the ID of the flow (i.e., the hash of its 5-tuple), the source and destination IP addresses, and the timestamp when the flow is detected. The notification sent by the data plane when a flow is terminated contains the flow ID, duration, total number of bytes sent, and total number of retransmissions.

After obtaining the ID of a flow, the control plane queries the register cells indexed by the ID. Because the control plane runs at the software speed and cannot extract per-packet measurements [13], it periodically queries the registers. Smaller extraction periods are preferred as they yield a higher granularity; however, they induce higher overhead. Experiments showed that the minimum possible extraction period is four milliseconds. For some metrics like the number of bytes and the number of packet losses, periodic extraction imposes no data loss as the accumulated value of the metric is targeted. On the other hand, metrics like RTT and queue occupancy should not be accumulated. The recent value of these metrics is the most representative. However, reporting the recent value during an extraction period means that all non-extracted values are discarded. This challenge is mitigated by implementing low-pass filters in the data plane and calculating the weighted moving average. The control plane then extracts the moving average of the measurements collected by the data plane during the extraction period.

After extracting the measurements, the control plane performs additional processing before reporting them to the archiver. For instance, the control plane calculates throughput by taking the total number of extracted bytes and dividing it by the extraction period. Additionally, the control plane derives measurements like link utilization and fairness among the TCP flows. Furthermore, the control plane selectively reports the measurements. The control plane only reports the measurements required to reflect the state of the flows. If a metric (e.g., RTT, throughput, etc.) can be linearly modeled, the control plane reduces its reporting rate (i.e., the rate of reporting measurements to the archiver). The adaptive model is the component responsible for dynamically adjusting the reporting rate.

The workflow of data extraction is depicted in Fig. 5. The data plane incorporates a flow tracker, implemented using count-min sketches and a match-action table. This tracker has four roles:

- Calculates the flow ID of an incoming packet.
- Identifies whether a packet belongs to an existing or new flow.
- Notifies the control plane if the packet belongs to a new flow.
- Detects the termination of flows and notify the control plane.

The control plane utilizes this tracker to update its list of active flows. The list contains the flow IDs used for querying the registers in the data plane. In the provided figure, two flows are depicted. The red flow showcases data transmission characterized by high fluctuations in throughput and occasional packet losses, yet maintaining a consistent RTT. Conversely, the blue flow represents data transmission under stable network conditions—no packet losses, consistent RTT, and a consistent variation in throughput.

The control plane extracts measurements for both flows from the registers. It then processes these measurements and appends necessary metadata for the archiving process. Afterward, the adaptive model determines which measurements should be reported to the archiver. For the red flow, the model decides to report all throughput and IAT measurements, discarding some RTT and packet loss measurements. For the blue flow, the adaptive model opts not to report any packet loss measurements, choosing only some throughput and RTT measurements for reporting, and reporting all IAT measurements.

Before discussing the adaptive model in details, it is important to define some key terminologies:

- Extraction rate refers to the constant rate at which measurements are pulled from the data plane to the control plane of the P4 PDP switch.
- Observations are the measurements retrieved from the data plane.
- Samples are the specific observations chosen by the model to be sent to the archiver for storage.
- Reports are the normalized samples prepared by the control plane for the archiver to ingest.
- Reporting rate, also known as sampling rate, is the rate at which reports are sent to the archiver. This rate is dynamically adjusted by the model.

4.4. Adaptive Model and Measurement Reporting

The adaptive model utilized by the proposed system is based on the LP method. In the LP method, the future value is predicted using the recent sample and the average rate of change for the last N samples as follows:

$$x_{p_{N+1}} = x_N + \frac{\Delta T_{current}}{N-1} \sum_{i=1}^{N-1} \frac{x_{i+1} - x_i}{t_{i+1} - t_i}$$

where:

- N is the number of samples considered.
- x_N is the most recent sample.
- x_i and t_i represent the value and arrival time of the i^{th} sample, respectively.
- $\Delta T_{current}$ is the time gap between the most recent sample and the next sample.
- $x_{p_{N+1}}$ is the predicted value for the next sample.

When x_{N+1} is sampled, the error in prediction m is calculated by dividing x_{N+1} over $x_{p_{N+1}}$ ($m = \frac{x_{N+1}}{x_{p_{N+1}}}$). If the error is close enough to unity (i.e., $|m - 1| \leq \epsilon$), the sampling rate is not adjusted as the linear model captured the traffic pattern. The rate is reduced or incremented depending on how far m is from unity. The method adjusts the sampling rate by configuring ΔT_{next} , the time between the next sample and the sample that follows. The method also defines parameter σ , which specifies how close to one m should be to consider the prediction accurate (i.e., the prediction is accurate if: $1 - \sigma \leq m \leq 1 + \sigma$).

The proposed model operates on the observations extracted from the data plane. Because the extraction rate is fixed, the time between any two consecutive observations (ΔT) is fixed. The expected value for each future observation is calculated using the rate of change rather than the average rate of change. The equation to calculate $x_{p_{N+1}}$ in the proposed model is:

$$\begin{aligned} x_{p_{N+1}} &= x_N + \frac{\Delta T}{N-1} \sum_{i=1}^{N-1} \frac{x_{i+1} - x_i}{\Delta T} \\ &= x_N + \frac{x_N - x_1}{N-1} \end{aligned}$$

Similar to LP, the model defines $\Delta T_{current}$ and ΔT_{next} variables to select the observations to be reported. The minimum and maximum values that ΔT_{next} can obtain are T_{min} and T_{max} . The model decides whether an observation should be reported on a per-observation basis. An observation is reported if the time elapsed between it and the last sampled observation is larger than $\Delta T_{current}$ or if the error m is not within σ from unity.

Table 1 presents the rules for adjusting ΔT_{next} based on $\Delta T_{current}$ and m . If m is within σ from unity, ΔT_{next} is set to the minimum of T_{max} and $\Delta T_{current} + 1$. If the error is not within the allowed boundaries, ΔT_{next} is set to T_{min} . T_{min} is equivalent to ΔT , or the time interval between two consecutive observations (i.e., the data reporting rate is set to the data extraction rate). T_{max} is configured depending on the maximum allowed Relative Mean Error (RME). RME represents the normalized absolute deviation between the mean of the sampled observations and the mean of all observations. The larger the value of T_{max} , the lower the number of reported observations at the cost of increasing the RME.

Table 1: Rules to define the next interval between samples in the proposed system.

Current m	ΔT_{next}
$1 - \sigma < m < 1 + \sigma$	$\min(T_{max}, \Delta T_{current} + 1)$
$m < 1 - \sigma$	T_{min}
$m > 1 + \sigma$	T_{min}

The upper bound on the RME increases with the increase in T_{max} because of the accumulation of errors. Even though each unreported observation has a minor error with respect to its predicted value, the accumulation of the minor errors might result in significant RME. The model stores the observations used to predict future values in vector x .

If the model fails to predict the behavior of the network, it discards the elements of x except x_{N+1} . x_{N+1} (which is x_1 after discarding the other observations), can be the first element in a new traffic pattern or an anomaly. In case there is a new traffic pattern, then all the previous observations are anomalies and should be discarded. On the other hand, if the network is still following the same pattern and x_{N+1} is the anomaly, then x_{N+1} will be automatically discarded in the next prediction iteration. After a failed prediction, the model maximizes the sampling rate (i.e., $\Delta T_{next} = T_{min}$) in order to detect the new pattern faster. To make the prediction more robust, the model waits for three iterations after the wrong prediction to make new predictions.

Another advantage of emptying x is that for any element x_i , the maximum and minimum values it can obtain are expressed in terms of x_1 :

$$\begin{aligned} x_{i_{max}} &= \frac{ix_1}{2(1-\sigma)^{(i-1)}} - x_1 \left(\sum_{j=1}^{i-2} \frac{i}{(i+1-j)(i-j)(1-\sigma)^j} \right) \\ x_{i_{min}} &= \frac{ix_1}{2(1+\sigma)^{(i-1)}} - x_1 \left(\sum_{j=1}^{i-2} \frac{i}{(i+1-j)(i-j)(1+\sigma)^j} \right) \end{aligned}$$

The difference d_i between any two consecutive observations x_{i-1} and x_i is bounded as follows:

$$\frac{\frac{x_{i-1}-x_1}{i-1} - \sigma x_{i-1}}{1+\sigma} \leq d_i \leq \frac{\frac{x_{i-1}-x_1}{i-1} + \sigma x_{i-1}}{1-\sigma}$$

The boundaries on the difference between element x_{N+1} and μ can then be expressed as follows:

$$\begin{aligned} x_{N+1} - \mu &\leq \frac{N}{2} \cdot \frac{\sigma x_{N_{max}}}{1-\sigma} + \frac{x_{N_{max}} - x_1}{1-\sigma} \\ x_{N+1} - \mu &\geq \frac{N}{2} \cdot \frac{\sigma x_{N_{min}}}{1+\sigma} + \frac{x_{N_{min}} - x_1}{1+\sigma} \end{aligned}$$

where x_{N+1} is the current observation and μ is the mean of the $N+1$ elements of x ($\mu = \sum_{i=1}^{N+1} \frac{x_i}{N+1}$). Thus, the upper bound on the absolute difference between x_{N+1} and μ can be expressed in terms of N, σ, x_1 .

Furthermore, the model can proactively adjust the sampling rate by only storing the most recent observation x_N . If the

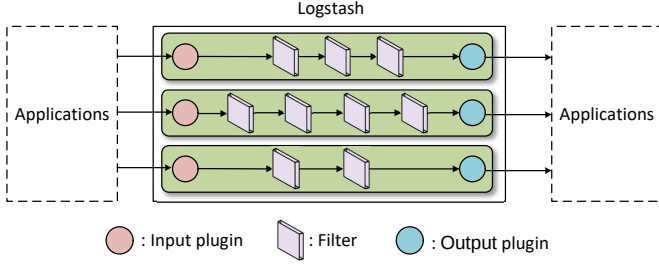


Figure 6: Logstash with three processing pipelines.

model fails to predict a close enough evaluation of the observation x_{N+1} , it reports both x_N and x_{N+1} . By reporting x_N , the model provides sufficient details to identify the old traffic pattern (i.e., the pattern followed by $\{x_1, \dots, x_N\}$).

Optimized measurement reporting for queue occupancy measurements: Linearly modeling queue occupancy measurements is not feasible as they rely on the behavior of multiple independent flows. For this, the proposed system identifies the optimal set of reports that minimizes the RME. To achieve fine-grain resolution, the control plane continuously queries the queue occupancy register and retrieves around 200 observations per second. After that, the control plane finds the optimal combination of reports that best describe the set of observed measurements.

Problem formulation: define x as the vector containing all the observations such that x_i is the i^{th} extracted observation and N to be the total number of observations. Define v such that $v_i \in \{0, 1\}$ for $i = 1, 2, \dots, N$. v is used to identify the elements of x to be reported such that element x_i is reported if and only if $v_i = 1$. Define $\lambda \in [0, 1]$ to be a hyperparameter representing the amount of penalizing on the number of reported samples. Define f to be the cost function. The function f is defined based on the RME and the number of reported samples. The goal is to find v^* that minimizes f (i.e., minimizes the RME and the penalty on the number of reported samples). The optimization problem can then be formulated as follows:

$$\min_v f = \frac{\left| \frac{v \cdot x}{\|v\|^2} - \mu \right|}{\mu} + \lambda \cdot \|v\|$$

The optimization problem is solved and v^* is calculated for each vector x . Element x_i is reported if $v_i^* = 1$ for $i = 1, 2, \dots, N$. All observations not reported by the control plane are discarded.

4.5. Measurements Storage and Visualization

The control plane of the P4 PDP switch forwards the measurements collected by the data plane to the perfSONAR archiver. The component responsible for receiving the measurements is Logstash.

As shown in Fig. 6, Logstash ingests data from multiple inputs, processes the data, and forwards it to multiple outputs. The data traverses from the input to the output components and passes through multiple filters. Each input-output path can have different filters depending on the processing logic. The data source can be hardware, software, or API calls, among others.

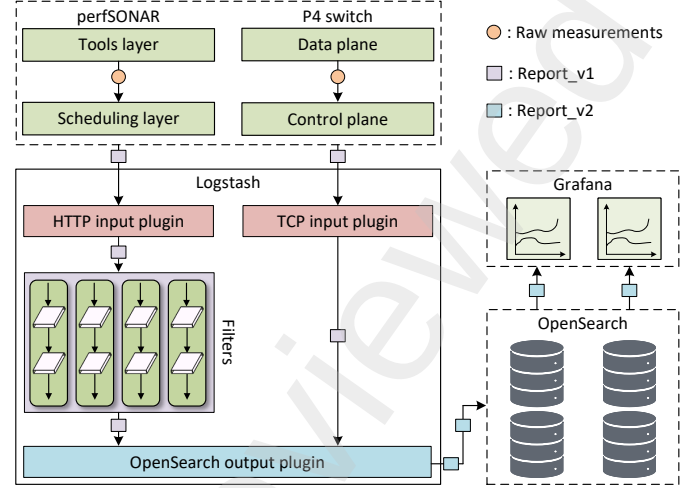


Figure 7: Measurement storage and visualization. The control plane formats the raw measurements extracted from the data plane to create structured reports (Report_v1). Logstash receives the reports via the TCP input plugin, adds the metadata required by the OpenSearch database, and forwards the new reports (Report_v2) to OpenSearch.

To support different sources, Logstash deploys different input plugins. It also has different output plugins to forward the processed data into different applications.

The APIs used by perfSONAR layers to communicate are REST-based. REST APIs communicate via the HTTP protocol. When a test is performed in perfSONAR, the tools layer forwards the measurements to the scheduling layer. The scheduling layer formats the data in JavaScript Object Notation (JSON) and forwards it to Logstash. The input plugin used to connect the scheduling layer and Logstash is the HTTP plugin. Because the tools in perfSONAR produce different measurements, Logstash has a dedicated input-output path for each tool. After receiving a report from the input plugin, Logstash processes it using its filters such that each filter's output is the next filter's input. The processed report is then forwarded to the OpenSearch output plugin. The plugin appends the metadata required by OpenSearch to ingest and store the report.

Similar to perfSONAR, the proposed system connects with Logstash to store the measurements. As depicted in Fig. 7, it utilizes the TCP input plugin. The plugin requires configuring the IP address and the port of the source application. In order to achieve a seamless integration between the PDP switch and perfSONAR, the control plane follows the same format adopted by perfSONAR scheduling layer. The control plane also performs the necessary processing on the reports, eliminating the need for any new filters in Logstash. The reports received from the PDP switch are directly forwarded from the TCP input plugin to the OpenSearch output plugin. The output plugin then adds the necessary metadata and stores the reports in OpenSearch.

After storing the data in OpenSearch, perfSONAR uses Grafana [23] to visualize the data. Grafana queries OpenSearch and provides the tools to customize data visualization. The user can control the frequency of queries, select the time interval to present the data (e.g., last five minutes), and distribute queries

Table 2: perfSONAR before and after the integration of PDP.

Metric	Without PDP	With PDP	Notes
Measurements type	Mainly active measurements	Mainly passive measurements	Unlike active measurements, passive measurements induce no overhead on the network
Measurements source	Injected traffic	Real traffic	Measurements not collected from the real traffic might not be representative
Granularity	Limited granularity	Per-flow and per-packet granularity	Tools utilized by the regular perfSONAR are not designed to support sub-millisecond granularity
Visibility	Limited by the active tests	Real-time visibility over all data transfers	perfSONAR with PDP provides visibility as long as there is a data transfer
Queue occupancy and IAT monitoring	Not supported	Supported (implemented use-cases include detecting microbursts and mmWave blockages)	Microburst and blockage detection require per-packet granularity
End-point failures	Not supported	Detects and reports limitations caused by the endpoints	Regular perfSONAR does not detect problems originated from the DTNs

on different graphs to provide a comprehensive overview of the stored data. Furthermore, Grafana allows the user to filter queries by the keys of the reports. The reports are JSON objects that store the data in the key-value pairs. This feature allows the user to define a new searchable field to the reports by appending new keys. For instance, the control plane can add the pair `{source: P4}` to all the reports, allowing the network administrator to filter the measurements reported by the PDP switch. The proposed system does not require any additional configuration on Grafana. Once the measurements are reported to OpenSearch, they can be viewed at run time using Grafana.

5. Features and Enhancements

The proposed system enhances the performance and problem detection and localization capabilities of perfSONAR. Table 2 compares perfSONAR with and without the integration of PDP. First, the integration eliminates the trade-off between the visibility of perfSONAR and the overhead it imposes on the network. By passively collecting measurements, the system provides visibility for the duration of the data transfers (i.e., a path is monitored as long as there is a data transfer on it). This significant increase in visibility does not add overhead to the network. The measurements are collected from the duplicated traffic and calculated at line rate using the PDP switch. There is no risk of increasing network latency or affecting the overall performance. Because the switch does not actively interfere, it is suitable for high-performance networks [24].

Unlike regular perfSONAR, which collects measurements from the injected traffic, perfSONAR integrated with PDP collects measurements from the real traffic. By having access to the real traffic and utilizing the PDP switch's customizable processing capabilities, perfSONAR achieves per-flow per-packet granularity. Per-flow monitoring is a functionality discussed as a future work by the original perfSONAR paper [25] but never implemented in perfSONAR because of the technology limitation. Per-flow measurements provide the necessary details required to analyze and understand science traffic, thus enhancing

the monitoring capabilities of perfSONAR. Besides, per-flow monitoring enhances the problem localization functionalities in perfSONAR. One use case is detecting if a flow is limited by the network or the sender/receiver by monitoring the bytes in flight (BIF). BIF is the number of bytes yet to be acknowledged by the receiver. In additive increase multiplicative decrease (AIMD) congestion control algorithms (CCA), this number increases if the TCP windows on the receiver and sender are large enough and no packet losses occur [26]. If the BIF stops increasing without packet losses, then the bottleneck is the sender/receiver (i.e., the DTNs in the context of Science DMZ).

Allowing perfSONAR to detect problems originating from the communicating entities guarantees that the active tests are performed only when necessary. In regular perfSONAR, active tests are performed after noticing a degradation in performance. Although active tests have an associated overhead on the available resources, they are necessary to locate and troubleshoot the networking problems. However, that is not the case when the degradation is caused by the DTNs. Because the DTNs are located outside the networking segment monitored by perfSONAR, active tests fail to detect problems originating from the DTNs. Thus, performing active tests adds overhead without helping networking engineers in locating the root cause of performance degradation.

perfSONAR integrated with PDP supports metrics that require per-packet granularity, like queue occupancy and packets IAT [27]. Through monitoring the queue occupancy, perfSONAR identifies the packet losses caused by bloated queues (also referred to as buffers). Because of the bursty nature of the traffic in Science DMZ [9], network devices are configured with large buffer sizes. If a single device is configured with a small-sized buffer and fails to absorb the bursts of traffic, the performance of the data transfers on the segment containing the bottleneck device will be degraded.

Monitoring packets IAT provides more statistics on the science traffic and allows the detection of problems caused by millimeter wave (mmWave) blockages in data centers. mmWave is a wireless technology that falls under the IEEE 802.11ad stan-

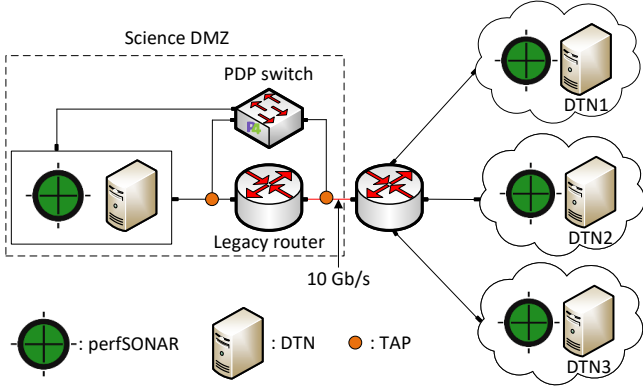


Figure 8: Topology used for the experiments.

dard [28]. It allows the transmission of multiple Gigabits per second (Gb/s) through beam forming and multiple input multiple output (MIMO) techniques. Wireless data center networking with mmWave links provides lower latency [29], lower energy consumption, and higher scalability [30, 31, 32]. However, connections over mmWave are sensitive to blockage, where human bodies can severely degrade the performance [5]. Monitoring packets IAT allows the detection of mmWave blockages, enhancing the problem detection and localization capabilities of perfSONAR.

6. Experimentation

This section evaluates the performance of perfSONAR integrated with PDP. It first highlights the monitoring capabilities of the proposed integration. Then, it shows how the added set of metrics (i.e., packets IAT and queue occupancy) enhances the problem detection and localization functionalities of perfSONAR. After that, the section evaluates the performance of the deployed adaptive system.

6.1. Topology Setup

The experimental topology is depicted in Fig. 8. There are four networks; one operates as the Science DMZ, and three operate as the WAN. Each network has one DTN node for data transfers and one perfSONAR node for monitoring. The Science DMZ and the WAN are connected via two Juniper MX204 routers [33] (legacy routers). The link between the two routers is the bottleneck with 10Gb/s capacity. Two passive optical TAPs are installed at the ingress and egress ports of the legacy router inside the Science DMZ. The TAPs can handle speeds up to 100Gb/s [34]. They duplicate the traffic to the PDP switch. The used PDP switch is Edgecore Wedge100BF-32X switch (Intel Tofino) [35]. Traffic is generated using the iPerf3 tool [36].

6.2. Per-flow Monitoring

This section shows the per-flow granularity of the proposed system. Three data transfers are performed between the Science DMZ and the WAN (one data transfer per external DTN).

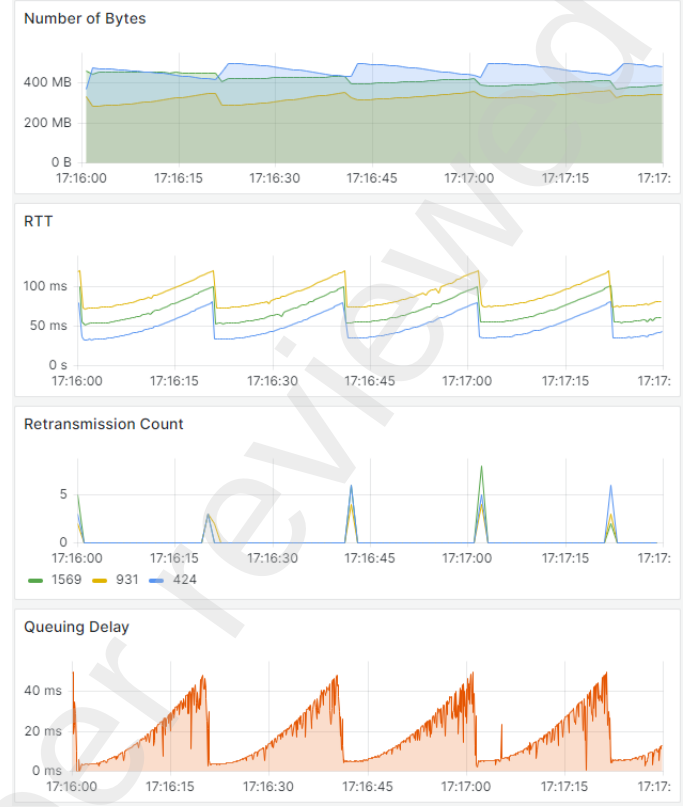


Figure 9: Measurements collected by the data plane. The number of bytes, RTT, and retransmission count are collected on a per-flow basis.

The RTTs between the Science DMZ and the three external networks are set to 30, 50, and 70 milliseconds (ms), respectively. The buffer size of the Science DMZ router is set to 50 Mbits or one bandwidth-delay product (1 BDP). Grafana is used to visualize the collected measurements in real-time. The figures depicting the results of the experiments are screen captures from the Grafana dashboard.

6.2.1. Metrics collected by the data plane

Fig. 9 depicts measurement graphs pertaining to (1) per-flow number of bytes; (2) RTT; (3) retransmission count; (4) queuing delay, labeled from top to bottom. The number of bytes and the retransmission count graphs show how the flows continuously increase their sending rate till packet loss occurs. As the flows increase their sending rate above the capacity of the bottleneck link, the queue builds up. The increase in the queuing delay affects the RTTs of the flows. The packets that cannot be held by the queue of the router (when the queue is full) are dropped, leading to packet retransmissions. The per-flow measurements allow the network administrator to analyze, understand, and diagnose the events occurring by the traffic.

6.2.2. Metrics derived by the control plane

Because the control plane has access to all the measurements collected by the data plane, it can derive new metrics. Fig. 10 depicts the metrics derived by the control plane. The throughput is derived from the number of bytes and the data extraction

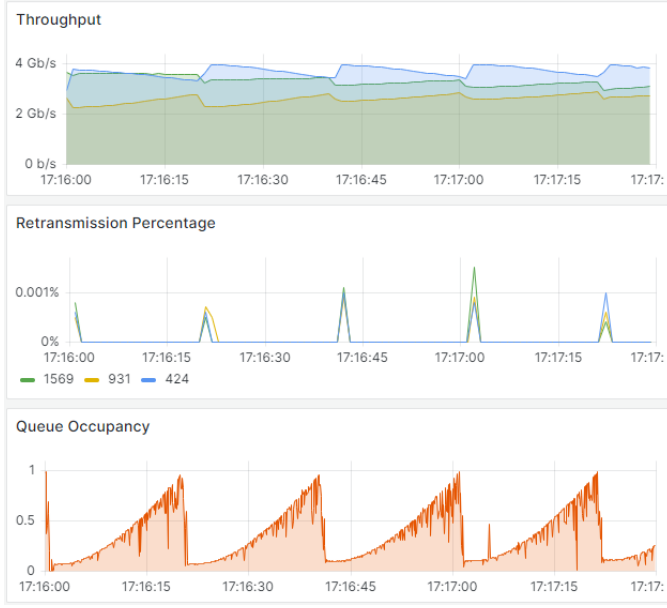


Figure 10: Metrics calculated in the control plane. Retransmission percentage is calculated by dividing the retransmission count by the total number of packets. Queue occupancy is calculated by dividing the queue delay by the maximum queue size.

duration. The retransmission percentage is derived from the retransmission count and the total number of packets. The queue occupancy is derived from the queuing delay and the queue size of the legacy router.

The control plane also calculates new metrics by combining measurements from multiple flows. As depicted in Fig. 11, the control plane reports the link utilization and the fairness among TCP flows. The link utilization is calculated by dividing the aggregated throughput of the flows by the bandwidth of the bottleneck link. The fairness is calculated based on Jain's fairness index [37] as follows:

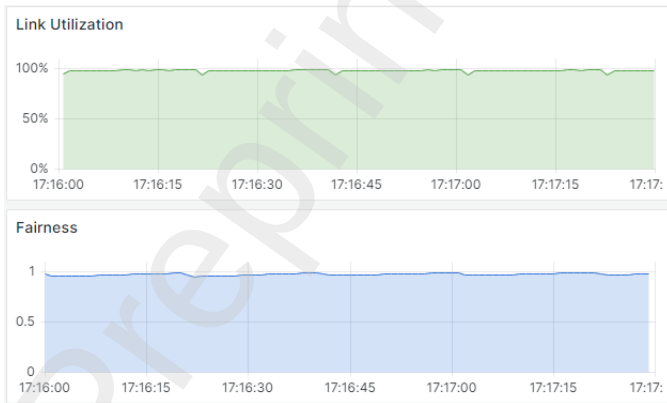


Figure 11: Metrics calculated in the control plane by combining measurements from multiple flows.

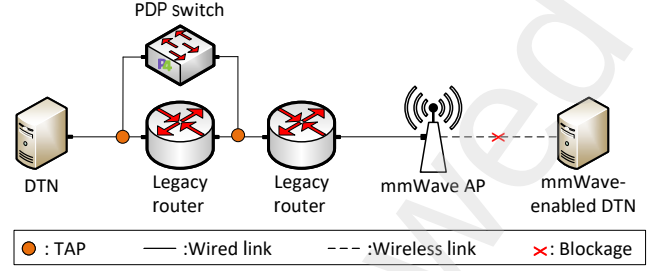


Figure 12: Testbed used to evaluate the performance of a PDP switch in detecting mmWave blockages.

$$F = \frac{\left(\sum_{i=1}^N x_i \right)^2}{N \cdot \sum_{i=1}^N x_i^2}$$

where:

- F is the Jain's fairness index.
- N is the number of flows.
- x_i is the resource allocation (e.g., bandwidth) for the i th flow.

The fairness index takes values between zero and one, such that the higher the value, the more fair the allocation of resources.

6.3. Problem Detection and Localization

This section evaluates how the proposed system can detect and localize network problems. In particular, two use cases are presented, namely mmWave blockages and small-sized buffers.

6.3.1. Detecting problems caused by mmWave blockages

The testbed used in this experiment is depicted in Fig. 12. The mmWave-enabled access point (AP) is Netgear NightHawk x10 [38]. The wireless link connecting the AP and the mmWave-enabled DTN operates over the IEEE 802.11ad standard [28]. The data transfer is occurring between the two DTNs. During the transfer, the mmWave wireless link between the AP and the DTN is blocked for 1 second. As shown in Fig. 13, the IAT of a few packets increases by multiple folds during the blockage. The P4 switch detects the blockage within 10ms. The switch is capable of detecting the blockage before the connection performance degrades [5].

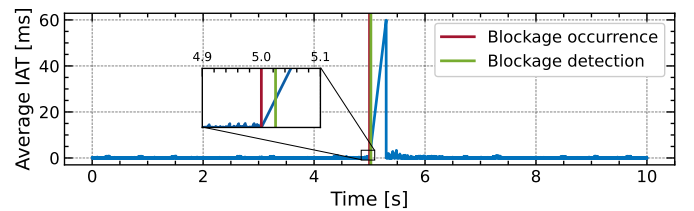


Figure 13: Blockage detection by monitoring the IAT.

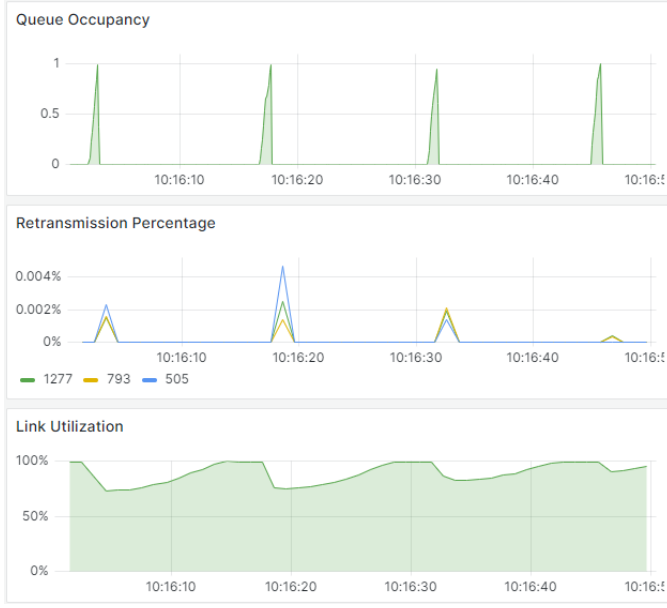


Figure 14: Detecting small-sized buffers.

6.3.2. Detecting problems caused by small-sized buffers

In this experiment, a small-sized buffer (BDP/4) is configured on the legacy router. Three data transfers are performed between the Science DMZ and the external DTNs. The *-pacing-timer* option of iperf3 tool is used to generate bursty traffic. The results of the experiment are depicted in Fig. 14. The queue fails to absorb the bursts of traffic, leading to packet losses. Frequent packet losses prevent the TCP flows from fully utilizing the available bandwidth, as shown by the link utilization graph. By monitoring the measurements collected by the PDP switch, network engineers can detect performance issues related to misconfigured buffer sizes.

It is important to note that visualizing microbursts in Grafana is challenging as it requires querying OpenSearch using a microsecond time scale, which overloads OpenSearch and Grafana daemons. Nevertheless, by maintaining the moving average of the queueing delay on the data plane, the effects of microbursts can be visualized.

6.3.3. Locating the root cause of connection limitation

In this experiment, three data transfers are performed between the Science DMZ and the three external DTNs. For the first data transfer, the TCP sending and receiving buffer sizes of DTN1 are reduced, emulating the scenario where the receiver is the bottleneck. For the second data transfer, the DTN in the Science DMZ is configured to send data at a 1Gb/s rate, emulating the scenario where the sender is the bottleneck. For the third data transfer, 0.001% packet loss is induced to the route between the Science DMZ and DTN3, emulating the scenario where the network is the bottleneck.

The results of the experiment are depicted in Fig. 15. The PDP switch reports that the data transfers destined for DTN1 and DTN2 (flows with IDs 710 and 1721) are restricted by the sender/receiver, and the data transfer destined for DTN3 (flow

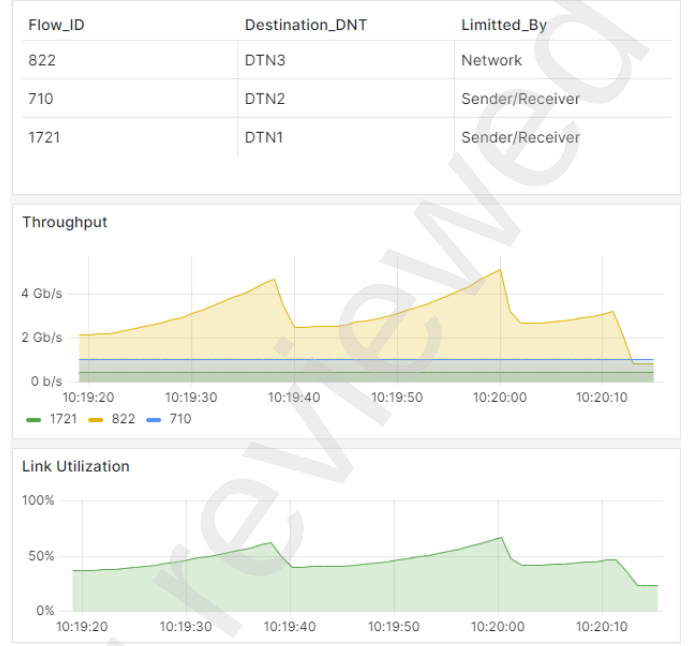


Figure 15: Reporting the flows limited by network and those limited by the sender/receiver.

with ID 822) is restricted by the network. The throughput graph shows the difference in behavior between the flows limited by the sender/receiver and the flow limited by the network. The flow restricted by the network continuously increases its data transferring rate till packet loss occurs. This approach increases bandwidth utilization. On the other hand, the flows restricted by the sender/receiver do not follow the same approach, underutilizing the available bandwidth (as shown by the link utilization graph).

6.4. Adaptive System

This section evaluates the performance of the deployed adaptive system. First, the impact of σ on the number of samples and the RME is evaluated. After that, the system is compared to two other adaptive systems.

6.4.1. Evaluating the Impact of σ

In this experiment, the performance of the adaptive system is compared with respect to different values of σ under severe network conditions (i.e., significant fluctuation in the throughput). Under stable network conditions, the adaptive system reduces the number of reported measurements by N_{max} times as long as σ is not extremely small (e.g., 0.001). Thus, the trade-off between the number of samples and the RME can only be evaluated under unstable network conditions. The used values of σ are 0.025, 0.05, 0.075, 0.1, 0.15, and 0.2. The number of reported samples and the RME are calculated for each value of σ .

The results of the experiment are depicted in Fig. 16. *Ground Truth Throughput* is the reported throughput with no sampling. With σ set to 0.025, the number of reports is 101, representing

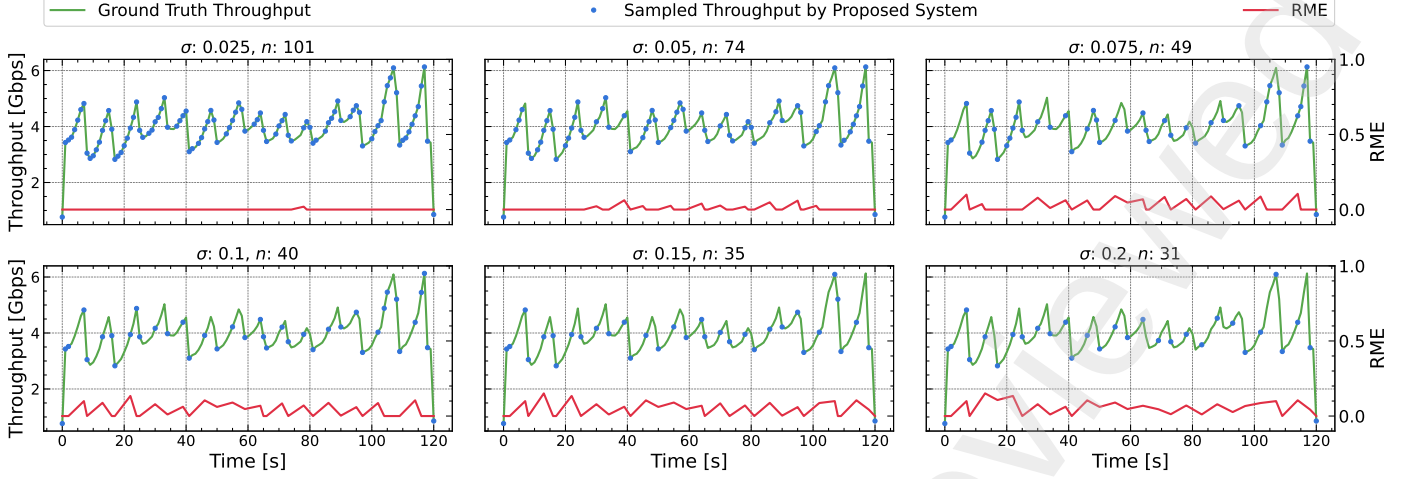


Figure 16: The number of samples and RME with respect to different values of σ .

a 18% reduction in the number of reported measurements compared to the *Ground Truth*. The RME is around zero throughout the test. With σ set to 0.05, the number of reports is 74, representing a 39% and 26% reduction in the number of reported measurements compared to the *Ground Truth* and 0.025 σ , respectively. The RME is slightly above zero for some of the reports. With σ set to 0.075, the RME significantly increases compared to 0.05 σ as the model did not report a few critical points (i.e., some of the peaks in the throughput graph are not reported). As the value of σ increases above 0.075, the RME becomes significantly large with a minor decrease in the number of samples.

6.4.2. Comparing with other adaptive systems

The proposed adaptive system is compared to two adaptive systems, namely, LP and MuST. MuST, or Multiadaptive sampling technique, is based on the LP method [39]. It dynamically adjusts the number of samples and the size of each sample by comparing the predicted value with the observed value. The rules to define ΔS_{next} , the size of the next sample and ΔT_{next} based on $\Delta S_{current}$, the current sample size, $\Delta T_{current}$, and m are summarized in table 3.

In this experiment, σ is set to 0.05 for the three systems. The results of the experiment are depicted in Fig. 17. The proposed system outperforms MuST and LP with respect to the sustained RME. The maximum observed RME is 0.02, and the average RME is 0.001. The proposed system is able to reflect the traffic pattern. MuST performed better than LP, where the maximum observed RME is 0.2, and the average RME is 0.02. MuST missed reporting a few critical measurements. For LP, the maximum observed RME is 0.27, and the average RME is 0.027. LP failed to reflect the traffic pattern in the interval [60, 80]. Regarding the number of samples, MuST has the least number of samples (49), followed by LP (52), followed by the proposed system (71). This outcome is expected as the proposed system increases the reporting rate if the model overestimates or underestimates the monitored metric. On the other hand, LP and MuST only increase the sampling rate if the model underesti-

Table 3: Rules to define the next interval between samples and the size of the next sample in MuST.

Current m	ΔT_{next}	ΔS_{next}
$1 - \sigma < m < 1 + \sigma$	$\Delta T_{current}$	$\Delta S_{current}$
$m < 1 - \sigma$	$m \Delta T_{current}$	$m \Delta S_{current}$
$m > 1 + \sigma$	$2 * \Delta T_{current}$	$1.15 \Delta S_{current}$

mates the metric.

Fig. 18 presents the cumulative distribution function (CDF) of the RME for the three adaptive systems. The results are obtained from running 20 data transfers. The proposed system outperforms MuST and LP, where 99% of the RME values are less than 0.05 and the maximum observed RME is 0.06. MuST performs better than LP, where 68% of the RME values are less than 0.05, 85% are less than 0.1, 90% are less than 0.15, 97% are less than 0.2, and the maximum observed RME is 0.26. LP has the higher overall RME, where 48% of the RME values are less than 0.05, 69% are less than 0.1, 85% are less than 0.15, 90% are less than 0.2, and the maximum observed RME is 0.28.

6.5. Troubleshooting Science DMZ at USC

The proposed system has been adopted by USC and is currently monitoring its Science DMZ. Under high utilization, the system reported significantly high RTT measurements. In order to identify the reason behind such observation, data transfers are conducted between USC's Science DMZ and Amazon Web Services (AWS) (USC has established a 10 Gbps connection, via Internet2, to AWS). For these data transfers, the total delay (propagation delay plus the queueing delay) was approximately up to 600ms. The data transfers were repeated when the network was not utilized and reported a total delay of 20ms. With this observation, the buffer size at the core switch was the main suspect. Most Science DMZs allocate a large buffer to key interfaces. At USC, the interface of the core switch facing the border router aggregates traffic from multiple ingress interfaces and thus must allocate memory for buffering packets. Non-optimal buffer sizing can lead to the following problems: over-

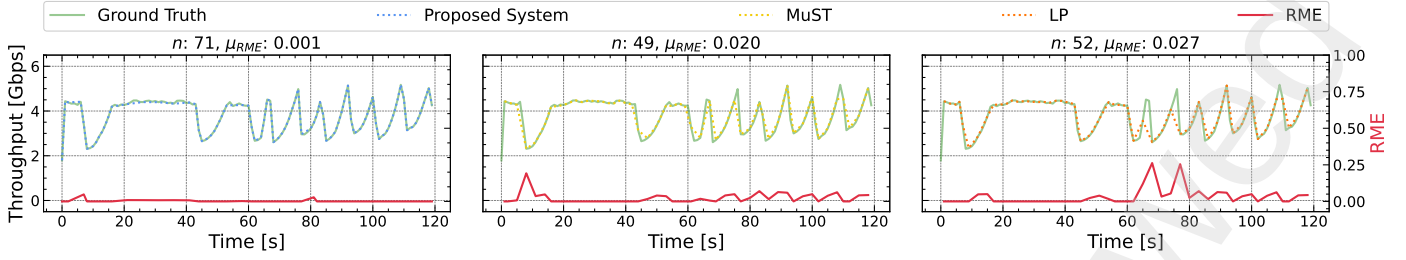


Figure 17: The throughput and RME comparison between the proposed system, MuST, and LP.

allocation produces excessive latency (a phenomenon known as buffer bloat), and under-allocation produces packet loss and poor link utilization.

After that, the buffer size of the core switch is hard coded according to the rule $B = \frac{c \cdot RTT_{av}}{\sqrt{N}}$. In this equation, B is the optimal buffer size, c is the link capacity of the egress interface, N is the number of flows crossing the switch, and RTT_{av} is the average RTT among all N flows. During this test, the science DMZ was only used for this flow (controlled environment). The results of tests with the large buffer and after adjusting the buffer size are depicted in Fig. 19. With a large buffer, the first flow (blue) grabbed more bandwidth throughout the data transfer time, resulting in a small fairness index. After adjusting the buffer size, the RTTs were minimized since the queueing delay was smaller and the fairness increased since the TCP reaction time was faster.

7. Discussion and Future Work

7.1. Data Aggregation

Periodic data extraction is essential to prevent the data plane from flooding the control plane with the per-packet measurements. All measurements not extracted by the control plane are discarded by the data plane. Data aggregation is performed by the data plane to minimize the loss of data caused by the periodic extraction. In the proposed system, the data extraction rate is constant. The control plane extracts the measurements based on a timer that determines the time gap between consecutive extractions. A better approach is for the data plane to decide which measurements to be extracted. This can be achieved by deploying an adaptive system directly on the data plane. The data plane monitors the variation in the new measurements with respect to the moving average of the aggregated measurements. The data plane keeps aggregating the measurements as long as

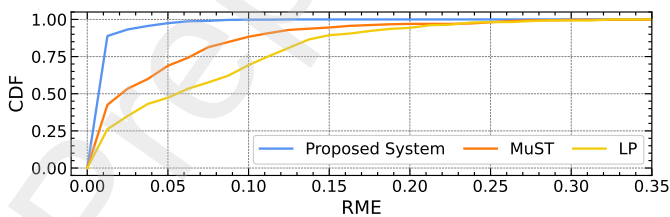


Figure 18: CDF of the RME for the proposed system, MuST, and LP.

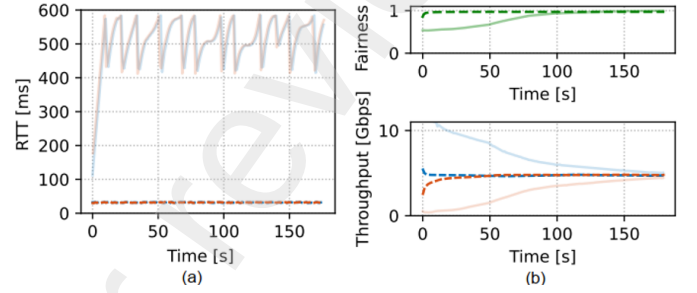


Figure 19: RTT results after adjusting the buffer size using two science flows from USC's Science DMZ to AWS. The shaded lines show the results with a large buffer. The RTTs are minimized since the queueing delay is smaller. (b) Throughput and fairness. The fairness increases since the TCP reaction time is faster.

no deviation in the traffic pattern is observed. If a deviation occurs, the data plane forwards the measurements directly to the archiver. Future work aims to offload the adaptive model to the data plane and connect the data plane directly to Logstash. A new set of Logstash filters will be defined to process the raw measurements from the data plane, as the control plane of the PDP switch will be eliminated from the data collection process.

7.2. Automation

perfSONAR monitors the routes connecting Science DMZ networks in order to verify that they operate as expected. After detecting a problem, networking engineers perform a set of active tests to locate the device or link that caused the issue. Despite the fact that this approach has proven successful in troubleshooting many problems [40], it has two drawbacks. The first drawback is the time required to detect the problem. perfSONAR depends on the active tests to detect the problems. The detection speed is constrained by the frequency of the tests, which is low for some metrics (e.g., throughput test is performed once every six hours [41]). The second drawback is the need for manual configuration of tests. perfSONAR is only utilized as a tool to assist engineers in locating the problems. It requires human intervention to configure the tests in order to locate the malfunctioning device. Automating the processes of problem detection and problem localization significantly reduces the time to perform them and eliminates human intervention.

One method to automate the detection process is through machine learning (ML). The per-flow measurements collected by

the PDP switch can be utilized to train an ML model to identify faulty traffic patterns. The model can be implemented on the control plane of the PDP switch, on the perfSONAR node, or on a stand-alone controller. Future work will investigate the optimal approach to implement the ML model.

Future work will also consider building a smart system that automatically localizes faulty devices. After the ML model detects a flow suffering from degradation in performance, the system traces the path followed by the flow to identify the faulty segment. After that, it utilizes the discovery layer of perfSONAR to locate perfSONAR nodes along the segment and automatically coordinates active tests to identify the link or device that caused the problem.

7.3. Dynamic Resource Allocation

A main challenge against offloading applications to the data plane is the limited on-chip resources [6]. To mitigate this challenge, the proposed system limits the number of flows the data plane can simultaneously monitor to 2048. This imposes no challenges when the switch is monitoring a single Science DMZ as the traffic is characterized by a few elephant flows [9]. Nevertheless, this limitation prevents the PDP switch from monitoring multiple Science DMZ networks concurrently even though it has the processing power (it can process 6.4 Terabits per second [42]).

One solution is to utilize FermatSketch, a dividable, additive, and subtractive data structure that dynamically allocates the resources of the data plane by considering the network state [43]. Future work aims to employ FermatSketch to maximize the number of flows that can be concurrently monitored. The system will allocate resources at run time for the victim flows (flows experiencing packet losses or significant fluctuation in the RTT and throughput). This decreases the amount of resources needed from being proportional to the number of active flows to the number of victim flows [43].

8. Related Work

NetSage is a data collection, analysis, and visualization platform [44]. It is designed to monitor the performance of a wide range of projects funded by the National Science Foundation (NSF). Netsage collects data from routers, switches, active testing sites, and science data archives through active and passive tools. The collected data are publicly accessible and published online (<https://portal.netsage.global/grafana>). The platform identifies the main sources and destinations for large data transfers, and analyzes longitudinal data to understand longer-term traffic behaviors [45].

Open Science Grid (OSG) is a comprehensive network monitoring platform that operates on the measurements collected by perfSONAR [46]. OSG collects the measurements by connecting to remote perfSONAR archivers and downloading all recent measurements. The collected measurements are stored at the University of Nebraska. This platform also has a centralized configuration system to configure the type of tests, the perfSONAR nodes participating in the tests, and the test schedule for the entire infrastructure.

Bezerra et al. [47] integrates PDP switches with Americas Lightpaths (AmLight) to provide sub-second granularity. AmLight is a project supported by the National Science Foundation International Research Network Connections (IRNC) program. It aims to improve cooperation between the U.S. and the nations of Latin America [48]. The PDP switches enabled In-band Network Telemetry (INT) in AmLight. In INT, the switches on a path append the measurements of interest (e.g., number of bytes received by a specific port) to the packet headers. The headers are then forwarded by the last switch on the path to a collector. INT allows AmLight to provide instantaneous bandwidth utilization, monitor packet drop probability and jitter, and trace packets.

The proposed system extends our two previous works [49, 50]. In [49], we enhanced perfSONAR measurement capabilities using P4 PDP switches. In [50], we defined a system to dynamically adjust the data collection mechanism based on the change in traffic patterns. The proposed system aims to improve the monitoring and problem localization functionalities of perfSONAR. It deploys a PDP switch that collects per-flow measurements from the real traffic in real time. By operating on real traffic and providing fine-grain measurements, the system enhances the accuracy of the traces collected by perfSONAR. Furthermore, the system implements a dynamic data collection mechanism to reduce the storage overhead, discarding redundant measurements and collecting representative traces. perfSONAR traces are used to understand longitudinal traffic behavior (e.g., NetSage and OSG), detect anomalies [51], and train models for network-wide anomaly detection [52]. Improving the accuracy of the traces positively affects the performance of all the applications and platforms that utilize the traces as a major data source.

9. Conclusion

This paper proposes integrating PDP with perfSONAR. A PDP switch is installed passively, collects measurements from the real traffic in real-time, and reports the measurements to the perfSONAR archiver. The integration enhances the granularity, visibility, accuracy, and troubleshooting capabilities of perfSONAR. Experiments revealed the capability of the proposed system to provide per-flow granular statistics in real-time. Additionally, mmWave blockages and small-sized buffers are presented as use cases to evaluate how the proposed system can detect and localize network problems. In order to minimize the storage and processing resources on the archiver, the paper implements an adaptive sampling model based on the LP method. The adaptive model dynamically adjusts the reporting rate based on the variation in the traffic.

Acknowledgments

This work was supported by the U.S. National Science Foundation, Office of Advanced Cyberinfrastructure, Award #2346726.

References

- [1] European Organization for Nuclear Research, Towards the future: tackling upcoming data challenges, [Online]. Available: <https://home.cern/about/computing> (2023).
- [2] E. Waltz, Portable DNA Sequencer MinION Helps Build the Internet of Living Things, [Online]. Available: <https://tinyurl.com/fysezkef> (2016).
- [3] E. Dart, L. Rotman, B. Tierney, M. Hester, J. Zurawski, The science DMZ: A network design pattern for data-intensive science, in: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis, 2013.
- [4] perfSONAR Project, What is perfSONAR?, [Online]. Available: <https://tinyurl.com/4xef95h9> (2023).
- [5] A. Mazloun, E. Kfoury, S. Sur, J. Crichigno, N. Ghani, Enhancing block-age detection and handover on 60 GHz networks with P4 programmable data planes, in: IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom), Istanbul, Turkey, 2023.
- [6] E. Kfoury, J. Crichigno, E. Bou-Harb, An exhaustive survey on P4 programmable data plane switches: Taxonomy, applications, challenges, and future trends, IEEE access (2021).
- [7] A. Mazloun, Source codes, [Online]. Available: <https://tinyurl.com/4y5xfun2>.
- [8] Crichigno, Jorge and Heyden, Andreas and Sagona, Paul and Kfoury, Elie, CC* Integration-Small: Enhancing Data Transfers by Enabling Programmability and Closed-loop Control in a Non-programmable Science DMZ, [Online]. Available: <https://tinyurl.com/mwennj5a> (2024).
- [9] J. Crichigno, E. Bou-Harb, N. Ghani, A comprehensive tutorial on science DMZ, IEEE Communications Surveys & Tutorials (2018).
- [10] Deploying a central measurement archive, [Online]. Available: <https://tinyurl.com/ehftk66w>.
- [11] Logstash, [Online]. Available: <https://www.elastic.co/logstash>.
- [12] Opensearch, [Online]. Available: <https://opensearch.org/>.
- [13] A. AlSabeh, J. Khoury, E. Kfoury, J. Crichigno, E. Bou-Harb, A survey on security applications of P4 programmable switches and a STRIDE-based vulnerability assessment, Computer Networks (2022).
- [14] J. Gomez, E. F. Kfoury, J. Crichigno, G. Srivastava, A survey on TCP enhancements using P4-programmable devices, Computer Networks (2022).
- [15] E. Kfoury, J. Crichigno, E. Bou-Harb, P4tune: Enabling programmability in non-programmable networks, IEEE Communications Magazine 61 (6) (2023) 132–138.
- [16] A. Mazloun, E. Kfoury, J. Gomez, J. Crichigno, A survey on rerouting techniques with P4 programmable data plane switches, Computer Networks (2023).
- [17] E. A. Hernandez, M. C. Chidester, A. D. George, Adaptive sampling for network management, Journal of Network and Systems Management 9 (2001) 409–434.
- [18] J. Gierl, J. Baca, F. Jakab, R. Andoga, Adaptive sampling in measuring traffic parameters in a computer network using a fuzzy regulator and a neural network, Cybernetics and Systems Analysis 44 (2008) 348–356.
- [19] Y. Lu, C. He, Resource allocation using adaptive linear prediction in WDM/TDM EPONs, AEU-International Journal of Electronics and Communications 64 (2) (2010) 173–176.
- [20] E. Kfoury, J. Crichigno, E. Bou-Harb, P4BS: Leveraging Passive Measurements from P4 Switches to Dynamically Modify a Router's Buffer Size, IEEE Transactions on Network and Service Management (2023).
- [21] E. F. Kfoury, S. Choueiri, A. Mazloun, A. AlSabeh, J. Gomez, J. Crichigno, A comprehensive survey on smartNICs: Architectures, development models, applications, and research directions, IEEE Access (2024).
- [22] X. Chen, H. Kim, J. M. Aman, W. Chang, M. Lee, J. Rexford, Measuring TCP round-trip time in the data plane, in: Proceedings of the Workshop on Secure Programmable Network Infrastructure, 2020, pp. 35–41.
- [23] Grafana Labs, The open observability platform, [Online]. Available: <https://grafana.com/> (2023).
- [24] B. Eriksson, P. Barford, R. Nowak, Network discovery from passive measurements, in: Proceedings of the ACM SIGCOMM 2008 conference on Data communication, 2008, pp. 291–302.
- [25] B. Tierney, J. Metzger, J. Boote, E. Boyd, A. Brown, R. Carlson, M. Zekauskas, J. Zurawski, M. Swamy, M. Grigoriev, perfSONAR: Instantiating a global network measurement framework, SOSP Wksp. Real Overlays and Distrib. Sys 28 (2009).
- [26] M. Ghasemi, T. Benson, J. Rexford, Dapper: Data plane performance diagnosis of TCP, in: Proceedings of the Symposium on SDN Research, 2017, pp. 61–74.
- [27] X. Chen, S. L. Feibish, Y. Koral, J. Rexford, O. Rottenstreich, S. Monetti, T. Wang, Fine-grained queue measurement in the data plane, in: Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, 2019, pp. 15–29.
- [28] T. Nitsche, C. Cordeiro, A. B. Flores, E. W. Knightly, E. Perahia, J. C. Widmer, IEEE 802.11 ad: directional 60 GHz communication for multi-Gigabit-per-second Wi-Fi, IEEE Communications Magazine 52 (12) (2014) 132–141.
- [29] Y. Katayama, K. Takano, Y. Kohda, N. Ohba, D. Nakano, Wireless data center networking with steered-beam mmwave links, in: 2011 IEEE wireless communications and networking conference, IEEE, 2011, pp. 2179–2184.
- [30] A. Hamza, J. Deogun, D. Alexander, Wireless communication in data centers: A survey, IEEE communications surveys & tutorials 18 (3) (2016) 1572–1595.
- [31] D. Halperin, S. Kandula, J. Padhye, P. Bahl, D. Wetherall, Augmenting data center networks with multi-gigabit wireless links, in: Proceedings of the ACM SIGCOMM 2011 conference, 2011, pp. 38–49.
- [32] H. Vardhan, N. Thomas, S. Ryu, B. Banerjee, R. Prakash, Wireless data center with millimeter wave network, in: 2010 IEEE global telecommunications conference GLOBECOM 2010, IEEE, 2010, pp. 1–6.
- [33] N. Screening, Juniper Networks MX204, [Online]. Available: <https://tinyurl.com/3exsks8m> (2022).
- [34] F. I. Sale, Optical TAP, [Online]. Available: <https://tinyurl.com/yc5j62f6> (2022).
- [35] Intel, Wedge 100BF-32X, 100GbE data center switch, Barefoot Networks, an Intel® company, [Online]. Available: <https://tinyurl.com/ye2y8jrj> (2023).
- [36] J. Dugan, S. Elliott, B. Mah, J. Poskanzer, K. Prabhu, iPerf3, tool for active measurements of the maximum achievable bandwidth on IP networks, URL: <https://github.com/esnet/iperf> (2014).
- [37] R. Jain, A. Durreli, G. Babic, Throughput fairness index: An explanation, in: ATM Forum contribution, Vol. 99, 1999.
- [38] Netgear nighthawk x10 smart WiFi router (ad7200), [Online]. Available: <https://tinyurl.com/4ufcvxwz>.
- [39] J. M. C. Silva, P. Carvalho, S. R. Lima, A multiadaptive sampling technique for cost-effective network measurements, Computer Networks 57 (17) (2013) 3357–3369.
- [40] Sample network issues discovered using perfsonar, [Online]. Available: <https://tinyurl.com/vucjp9sr>.
- [41] perfSONAR, perfSONAR Toolkit 5.0.7 documentation, [Online]. Available: <https://tinyurl.com/nhc5nx2v>.
- [42] Intel, Intel® Tofino 6.4 Tbps, 4 pipelines, [Online]. Available: <http://tinyurl.com/525sb3z8>.
- [43] K. Yang, Y. Wu, R. Miao, T. Yang, Z. Liu, Z. Xu, R. Qiu, Y. Zhao, H. Lv, Z. Ji, et al., Chameleon: Shifting Measurement Attention as Network State Changes, arXiv preprint arXiv:2301.00615 (2023).
- [44] K. Turner, M. Khanal, T. Seto-Mook, A. Gonzalez, J. Leigh, A. Lake, S. Baveha, S. Faci, B. Tierney, D. Doyle, The NetSage Measurement Framework: Design, Development, and Discoveries, in: 2020 IEEE/ACM Innovating the Network for Data-Intensive Science (INDIS), IEEE, 2020, pp. 45–56.
- [45] A. Gonzalez, J. Leigh, S. Peisert, B. Tierney, E. Balas, P. Radulovic, J. M. Schopf, Big data and analysis of data transfers for international research networks using NetSage, in: 2017 IEEE International Congress on Big Data (BigData Congress), IEEE, 2017, pp. 344–351.
- [46] M. Babik, S. McKee, B. Bockelman, E. Hernandez, E. Martelli, I. Vukotic, D. Weitzel, M. Zvada, Improving WLCG networks through monitoring and analytics, in: EPJ Web of Conferences, Vol. 214, EDP Sciences, 2019, p. 08006.
- [47] J. Bezerra, I. Brito, A. Quintana, J. Ibarra, V. Chergarova, R. Frez, H. Morgan, M. LeClerc, A. Paneri, Deploying per-packet telemetry in a long-haul network: the AmLight use case, in: 2021 IEEE Workshop on Innovating the Network for Data-Intensive Science (INDIS), IEEE, 2021, pp. 44–49.
- [48] J. Ibarra, J. Bezerra, H. Morgan, L. F. Lopez, D. A. Cox, M. Stanton,

- I. Machado, E. Grizendi, Benefits brought by the use of OpenFlow/SDN on the AmLight intercontinental research and education network, in: 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), IEEE, 2015, pp. 942–947.
- [49] A. Mazloun, J. Gomez, E. Kfoury, J. Crichigno, Enhancing perfSONAR Measurement Capabilities using P4 Programmable Data Planes, in: Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis, 2023, pp. 819–829.
- [50] A. Mazloun, A. AlSabe, E. Kfoury, J. Crichigno, perfSONAR: Enhancing Data Collection through Adaptive Sampling, in: NOMS 2024-2024 IEEE Network Operations and Management Symposium, IEEE, 2024, pp. 1–6.
- [51] P. Calyam, J. Pu, W. Mandrawa, A. Krishnamurthy, Ontimedetect: Dynamic network anomaly notification in perfSONAR deployments, in: 2010 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems, IEEE, 2010, pp. 328–337.
- [52] Y. Zhang, S. Debroy, P. Calyam, Network-wide anomaly event detection and diagnosis with perfSONAR, IEEE Transactions on Network and Service Management 13 (3) (2016) 666–680.